Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №8

“Мережеве та асинхронне програмування. REST API”

Виконав:
студент 3 курсу
групи ФЕП-33с
Качмар Денис

Викладач:
Проф. Свелеба Сергій Андрійович

Львів - 2026

ХІД РОБОТИ

Тема: Мережеве та асинхронне програмування. REST API

Мета: Ознайомитися з основами мережевого програмування у Python, принципами роботи протоколу HTTP, архітектурним стилем REST та форматом JSON. Навчитися створювати REST API за допомогою FastAPI, описувати та перевіряти структуру даних за допомогою Pydantic, використовувати асинхронне програмування async/await та asyncio, виконувати декілька операцій паралельно за допомогою Task і gather(), встановлювати обмеження часу виконання операцій, реалізовувати повторні спроби запитів та обмежувати кількість одночасних операцій за допомогою Semaphore. Навчитися використовувати бібліотеку HTTPX для взаємодії з REST API та зовнішніми HTTP-сервісами.

Варіант: 5 (Каталог автомобілів)

Постановка задачі:

1. Розширити проєкт свого варіанта з попередніх лабораторних робіт («Каталог автомобілів») та створити для нього асинхронний REST API. 2. Реалізувати роботу з ресурсами `cars` (автомобілі) та `manufacturers` (виробники): - `GET /cars` — отримання списку авто з фільтрацією (бренд, ціновий діапазон, пробіг, рік) та пагінацією; - `GET /cars/{id}` — отримання детальної інформації про автомобіль за ID; - `POST /cars` — додавання нового автомобіля до каталогу; - `PATCH /cars/{id}` — часткове оновлення ціни, пробігу або інших характеристик; - `DELETE /cars/{id}` — видалення автомобіля; - `GET /manufacturers/{id}/cars` — отримання списку автомобілів конкретного автовиробника; - Додаткові аналітичні ендпоінти: `GET /cars/analytics/summary` (найдорожче авто, авто з найменшим пробігом, середня ціна, загальна вартість). 3. Використовувати Pydantic-схеми для валідації вхідних даних (Request body) та серіалізації вихідних відповідей (Response models). 4. Забезпечити підтримку коректних HTTP статус-кодів (200 OK, 201 Created, 204 No Content, 400 Bad Request, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 504 Gateway Timeout). 5. Реалізувати асинхронні сценарії для конкурентного завантаження та агрегації групи автомобілів: - використання `async def` та `await`; - створення та конкурентне планування через `asyncio.Task`; - конкурентне виконання через `asyncio.gather()`; - обмеження одночасних HTTP-з'єднань через `asyncio.Semaphore`; - налаштування таймауту (`timeout`) за допомогою `asyncio.wait_for`; - механізм повторних спроб (`retry`) із прогресивною паузою (exponential backoff); - виконання асинхронних HTTP-запитів за допомогою `httpx.AsyncClient`. 6. Забезпечити повну інтеграцію мережевого шару з Persistence Layer (SQLite + SQLAlchemy ORM Repositories із Лабораторної №7) за допомогою механізму FastAPI Dependency Injection (`Depends(get_session)`). 7. Розробити набір автоматизованих інтеграційних тестів для REST API з використанням `fastapi.testclient.TestClient` та `httpx.AsyncClient` (з `ASGITransport`).

---

1. Опис REST API та архітектури системи

У межах лабораторної роботи №8 архітектура навчального проєкту «Каталог автомобілів» була доповнена мережевим рівнем взаємодії (Networking Layer), що перетворило монолітний застосунок у повноцінний розподілений веб-сервіс із RESTful API.

Загальна багаторівнева архітектура системи має вигляд:

```
HTTP Client (Browser / Swagger UI / curl / Mobile / httpx.AsyncClient)
    │
    │  HTTP Requests (GET, POST, PATCH, DELETE) + JSON Payload
    ▼
FastAPI REST API Layer (api.py)
    │
    │  Data Validation, Parsing & Serialization
    ▼
Pydantic Schemas Layer (schemas.py)
    │
    │  Business Logic & Validation
    ▼
Service Layer (services.py / core domains)
    │
    │  Data Access Abstraction (Depends(get_session))
    ▼
Repository Layer (repositories.py - SqlAlchemyCarRepository,
SqlAlchemyManufacturerRepository)
    │
    │  ORM & SQL Generation (SQLAlchemy ORM Models)
    ▼
Database (SQLite / In-Memory Engine)
```

Асинхронний конвеєр агрегації та обробки пакетних запитів побудований за такою схемою:

```
Async Batch Endpoint (POST /cars/batch або POST /external/batch-fetch)
    │
    ├── Task 1: fetch_single_resource_with_retry() ──┐
    ├── Task 2: fetch_single_resource_with_retry() ──┼──► asyncio.Semaphore(rate_limit)
──► httpx.AsyncClient
    └── Task 3: fetch_single_resource_with_retry() ──┘          │
                                                                ▼
                                                  Timeout & Retry Backoff
                                                                │
                                                                ▼
                                                  asyncio.gather(*tasks)
                                                                │
                                                                ▼
                                                  Aggregated JSON Response
```

Основні принципи реалізації: - **Statelessness (Відсутність збереження стану сесії)**: кожен HTTP-запит клієнта містить усю необхідну інформацію для його обробки сервером. - **Resource-Oriented Design (Орієнтованість на ресурси)**: доступ до сутностей організовано через однозначні URI іменників у множині (`/cars` , `/manufacturers`). - **Синхронно-асинхронний**

міст: робота з базою даних SQLite/SQLAlchemy здійснюється через сесії, керовані генератором-залежністю `get_session`, тоді як мережеві виклики та агрегація зовнішніх ресурсів виконуються повністю асинхронно за допомогою `asyncio` та `httpx.AsyncClient`.

---

2. Перелік endpoints та таблиця HTTP methods

REST API системи «Каталог автомобілів» реалізує повний життєвий цикл CRUD для сутностей автомобілів та виробників, а також розширені аналітичні та асинхронні пакетні операції.

Метод	Шлях (Endpoint)	Призначення	Тіло запиту	Статус-код успіху
`GET`	`/`	Перевірка працездатності сервісу (health check)	Відсутнє	`200 OK`
`GET`	`/cars`	Отримання списку авто з фільтрацією та пагінацією	Відсутнє (Query params)	`200 OK`
`GET`	`/cars/{car_id}`	Отримання інформації про автомобіль за його ID	Відсутнє (Path param)	`200 OK`
`POST`	`/cars`	Створення нового автомобіля	`CarCreate` (JSON)	`201 Created`
`PATCH`	`/cars/{car_id}`	Часткове оновлення атрибутів автомобіля	`CarUpdate` (JSON)	`200 OK`
`DELETE`	`/cars/{car_id}`	Видалення автомобіля за ID	Відсутнє (Path param)	`204 No Content`
`GET`	`/manufacturers`	Отримання списку всіх автовиробників	Відсутнє	`200 OK`
`POST`	`/manufacturers`	Реєстрація нового виробника	`ManufacturerCreate` (JSON)	`201 Created`
`GET`	`/manufacturers/{id}`	Отримання даних виробника за ID	Відсутнє (Path param)	`200 OK`
`GET`	`/manufacturers/{id}/cars`	Отримання списку авто заданого бренду	Відсутнє (Path param)	`200 OK`
`DELETE`	`/manufacturers/{id}`	Видалення виробника (із	Відсутнє (Path param)	`204 No Content`

Метод	Шлях (Endpoint)	Призначення	Тіло запиту	Статус-код успіху
		каскадом авто)		
GET	/cars/analytics/summary	Зведена статистика каталогу (min/max/avg)	Відсутнє	200 OK
GET	/manufacturers/analytics/statistics	Статистика по кожному автовиробнику	Відсутнє	200 OK
POST	/cars/batch	Конкурентне отримання групи авто за списком ID	BatchFetchRequest (JSON)	200 OK
POST	/external/batch-fetch	Асинхронний запит до зовнішніх URL з лімітом	RemoteBatchRequest (JSON)	200 OK

---

3. Приклади Path Parameters та Query Parameters

3.1. Path Parameters (Параметри шляху)

Параметри шляху ідентифікують конкретний екземпляр ресурсу в ієрархії URI:

- `GET /cars/5` — запит на отримання детальної інформації про автомобіль з первинним ключем `id = 5`.
- `PATCH /cars/3` — застосування часткових оновлень до авто з `id = 3`.
- `DELETE /cars/3` — видалення авто з `id = 3`.
- `GET /manufacturers/2/cars` — вибірка всіх автомобілів, що належать автовиробнику з `id = 2`.

Реалізація у FastAPI:

```python
@app.get("/cars/{car_id}", response_model=CarResponse)
async def get_car(
    car_id: int,
    session: Session = Depends(get_session),
) -> CarResponse:
    repo = SqlAlchemyCarRepository(session)
    car = repo.get_by_id(car_id)
    if car is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Car with ID {car_id} not found",
        )
    return CarResponse.model_validate(car)
```

3.2. Query Parameters (Параметри запиту)

Параметри запиту передаються після символу `?` і використовуються для фільтрації, сортування та пагінації списків:

- `GET /cars?brand=Audi&min_price=20000&max_price=50000` — фільтрація за брендом та ціновим діапазоном;
- `GET /cars?year=2021&max_mileage=60000` — пошук автомобілів 2021 року з пробігом до 60 000 км;
- `GET /cars?skip=10&limit=5` — пагінація: пропуск перших 10 записів та повернення наступних 5.

Реалізація у FastAPI з типізацією та значеннями за замовчуванням:

```python
@app.get("/cars", response_model=list[CarResponse])
async def list_cars(
    brand: str | None = None,
    min_price: float | None = Query(default=None, ge=0.0),
    max_price: float | None = Query(default=None, ge=0.0),
    max_mileage: float | None = Query(default=None, ge=0.0),
    year: int | None = Query(default=None, ge=1886, le=2100),
    skip: int = Query(default=0, ge=0),
    limit: int = Query(default=100, ge=1, le=500),
    session: Session = Depends(get_session),
) -> list[CarResponse]:
    repo = SqlAlchemyCarRepository(session)
    cars = repo.get_all(skip=skip, limit=limit)
    if brand:
        cars = [c for c in cars if c.manufacturer.name.lower() == brand.lower()]
    if min_price is not None:
        cars = [c for c in cars if c.price >= min_price]
    if max_price is not None:
        cars = [c for c in cars if c.price <= max_price]
    if max_mileage is not None:
        cars = [c for c in cars if c.mileage <= max_mileage]
    if year is not None:
        cars = [c for c in cars if c.year == year]
    return [CarResponse.model_validate(c) for c in cars]
```

4. Pydantic Schemas та правила валідації (Validation Rules)

Для валідації структури даних, строгої типізації та автоматичної серіалізації створено модуль `schemas.py` на базі бібліотеки **Pydantic V2**.

4.1. Схеми виробників (Manufacturers)

```
class ManufacturerCreate(BaseModel):
    name: str = Field(min_length=2, max_length=100)
    country: str = Field(min_length=2, max_length=100)

class ManufacturerResponse(BaseModel):
    id: int
    name: str
    country: str
    created_at: datetime
    model_config = ConfigDict(from_attributes=True)
```

4.2. Схеми автомобілів (Cars)

```
class CarCreate(BaseModel):
    manufacturer_id: int = Field(gt=0)
    model: str = Field(min_length=1, max_length=100)
    year: int = Field(ge=1886, le=2100)
    price: float = Field(ge=0.0)
    mileage: float = Field(ge=0.0)
    vin: str | None = Field(default=None, max_length=17)

    @field_validator("vin")
    @classmethod
    def validate_vin(cls, value: str | None) -> str | None:
        if value is None:
            return None
        cleaned = value.strip().upper()
        if len(cleaned) != 17:
            raise ValueError("VIN must contain exactly 17 characters")
        return cleaned

class CarUpdate(BaseModel):
    model: str | None = Field(default=None, min_length=1, max_length=100)
    year: int | None = Field(default=None, ge=1886, le=2100)
    price: float | None = Field(default=None, ge=0.0)
    mileage: float | None = Field(default=None, ge=0.0)
    vin: str | None = Field(default=None, max_length=17)

class CarResponse(BaseModel):
    id: int
    manufacturer_id: int
    manufacturer_name: str | None = None
    model: str
    year: int
    price: float
    mileage: float
    vin: str | None = None
    created_at: datetime
    model_config = ConfigDict(from_attributes=True)
```

4.3. Схеми аналітики та пакетних асинхронних запитів

```
class AnalyticsSummaryResponse(BaseModel):
    total_cars: int
    average_price: float
    total_inventory_value: float
    most_expensive_car: CarResponse | None = None
    lowest_mileage_car: CarResponse | None = None

class BatchFetchRequest(BaseModel):
    car_ids: list[int] = Field(min_length=1, max_length=50)

class BatchFetchResponse(BaseModel):
    total_requested: int
    successful_count: int
    failed_count: int
    cars: list[CarResponse]
    missing_ids: list[int]
```

4.4. Правила валідації (Validation Rules)

1. **Рядки**: `min_length=1..2` виключає порожні рядки або рядки з одного символу; `max_length=100` обмежує надмірні розміри вхідних полів.
2. **Числа**: - `gt=0` для зовнішніх ключів (`manufacturer_id > 0`); - `ge=0.0` для ціни та пробігу (відкидаються від'ємні значення з кодом `422 Unprocessable Entity`); - `ge=1886, le=2100` для року випуску (від першого авто Карла Бенца до майбутнього століття).
3. **Кастомний валідатор** `@field_validator("vin")`: перевіряє фіксовану довжину 17 символів та переводить VIN у верхній регістр.
4. **Конфігурація** `from_attributes = True`: забезпечує прозору серіалізацію ORM-об'єктів SQLAlchemy без ручного копіювання полів.

---

5. Код FastAPI Application

Головний файл веб-додатка `src/car_catalog/api.py` налаштовує роутинг, глобальні залежності, обробники CRUD та асинхронні точки доступу:

```
from collections.abc import Generator
from fastapi import Depends, FastAPI, HTTPException, Query, status
from sqlalchemy.orm import Session
from car_catalog.database import get_session_factory
from car_catalog.db_models import CarModel, ManufacturerModel
from car_catalog.external_api import fetch_resources_concurrently
from          car_catalog.repositories          import          SqlAlchemyCarRepository,
SqlAlchemyManufacturerRepository
from car_catalog.schemas import (
    AnalyticsSummaryResponse, BatchFetchRequest, BatchFetchResponse,
    CarCreate, CarResponse, CarUpdate, ManufacturerCreate,
    ManufacturerResponse, ManufacturerStatisticsResponse,
    RemoteBatchRequest, RemoteBatchResponse,
)

def get_session() -> Generator[Session, None, None]:
    factory = get_session_factory()
    session = factory()
    try:
        yield session
    finally:
        session.close()

app = FastAPI(
    title="Car Catalog REST API",
    description="Asynchronous REST API for Vehicle Inventory Management",
    version="0.8.0",
)
```

5.1. Реалізація GET

```
@app.get("/cars/{car_id}", response_model=CarResponse)
async def get_car(car_id: int, session: Session = Depends(get_session)) -> CarResponse:
    repo = SqlAlchemyCarRepository(session)
    car = repo.get_by_id(car_id)
    if car is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Car with ID {car_id} not found",
        )
    return CarResponse.model_validate(car)
```

5.2. Реалізація POST

```
@app.post("/cars", response_model=CarResponse, status_code=status.HTTP_201_CREATED)
async def create_car(data: CarCreate, session: Session = Depends(get_session)) ->
CarResponse:
    car_repo = SqlAlchemyCarRepository(session)
    manuf_repo = SqlAlchemyManufacturerRepository(session)
    if manuf_repo.get_by_id(data.manufacturer_id) is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Manufacturer with ID {data.manufacturer_id} does not exist",
        )
    if data.vin and car_repo.find_by_vin(data.vin):
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail=f"Car with VIN {data.vin} already exists",
        )
    new_car = CarModel(
        manufacturer_id=data.manufacturer_id,
        model=data.model,
        year=data.year,
        price=data.price,
        mileage=data.mileage,
        vin=data.vin,
    )
    saved = car_repo.add(new_car)
    session.commit()
    return CarResponse.model_validate(saved)
```

5.3. Реалізація PATCH

```
@app.patch("/cars/{car_id}", response_model=CarResponse)
async def update_car(car_id: int, data: CarUpdate, session: Session =
Depends(get_session)) -> CarResponse:
    repo = SqlAlchemyCarRepository(session)
    car = repo.get_by_id(car_id)
    if car is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Car with ID {car_id} not found",
        )
    updates = data.model_dump(exclude_unset=True)
    if "vin" in updates and updates["vin"] is not None:
        existing = repo.find_by_vin(updates["vin"])
        if existing and existing.id != car_id:
            raise HTTPException(
                status_code=status.HTTP_409_CONFLICT,
                detail=f"Car with VIN {updates['vin']} already exists",
            )
    for field, value in updates.items():
        setattr(car, field, value)
    session.commit()
    session.refresh(car)
    return CarResponse.model_validate(car)
```

5.4. Реалізація DELETE

```
@app.delete("/cars/{car_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_car(car_id: int, session: Session = Depends(get_session)) -> None:
    repo = SqlAlchemyCarRepository(session)
    deleted = repo.delete_by_id(car_id)
    if not deleted:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Car with ID {car_id} not found",
        )
    session.commit()
```

---

6. Приклади JSON запитів (Request) та відповідей (Response)

6.1. Створення автомобіля (POST /cars)

HTTP Request:

```
POST /cars HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/json

{
  "manufacturer_id": 1,
  "model": "RS6 Avant",
  "year": 2023,
  "price": 125000.0,
  "mileage": 12000.0,
  "vin": "WAUZZZF27PN012345"
}
```

HTTP Response:

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": 11,
  "manufacturer_id": 1,
  "manufacturer_name": "Audi",
  "model": "RS6 Avant",
  "year": 2023,
  "price": 125000.0,
  "mileage": 12000.0,
  "vin": "WAUZZZF27PN012345",
  "created_at": "2026-10-24T15:30:00"
}
```

6.2. Часткове оновлення ціни (PATCH /cars/11)

HTTP Request:

```
PATCH /cars/11 HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/json

{
  "price": 118000.0,
  "mileage": 14500.0
}
```

HTTP Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 11,
  "manufacturer_id": 1,
  "manufacturer_name": "Audi",
  "model": "RS6 Avant",
  "year": 2023,
  "price": 118000.0,
  "mileage": 14500.0,
  "vin": "WAUZZZF27PN012345",
  "created_at": "2026-10-24T15:30:00"
}
```

6.3. Помилка валідації Pydantic (POST /cars з некоректними даними)

HTTP Request:

```
POST /cars HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/json

{
  "manufacturer_id": 1,
  "model": "A4",
  "year": 1800,
  "price": -500.0,
  "mileage": -100.0
}
```

HTTP Response:

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

{
  "detail": [
    {
      "type": "greater_than_equal",
      "loc": ["body", "year"],
      "msg": "Input should be greater than or equal to 1886"
    },
    {
      "type": "greater_than_equal",
      "loc": ["body", "price"],
      "msg": "Input should be greater than or equal to 0"
    },
    {
      "type": "greater_than_equal",
      "loc": ["body", "mileage"],
      "msg": "Input should be greater than or equal to 0"
    }
  ]
}
```

6.4. Зведена аналітика каталогу (GET /cars/analytics/summary)

HTTP Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "total_cars": 10,
  "average_price": 50100.0,
  "total_inventory_value": 501000.0,
  "most_expensive_car": {
    "id": 4,
    "manufacturer_id": 2,
    "manufacturer_name": "BMW",
    "model": "M5",
    "year": 2022,
    "price": 105000.0,
    "mileage": 15000.0,
    "vin": "WBA53AH07NH987654",
    "created_at": "2026-10-24T15:00:00"
  },
  "lowest_mileage_car": {
    "id": 8,
    "manufacturer_id": 4,
    "manufacturer_name": "Porsche",
    "model": "911 GT3",
    "year": 2023,
    "price": 185000.0,
    "mileage": 3500.0,
    "vin": "WP0AB2A99PS112233",
    "created_at": "2026-10-24T15:00:00"
  }
}
```

---

7. Асинхронне програмування: async/await, Task, gather, timeout, retry, Semaphore, HTTPX

Модуль `src/car_catalog/external_api.py` реалізує повнофункціональний асинхронний мережевий рушій відповідно до індивідуального завдання Варіанта 5:

```
import asyncio
import httpx

async def fetch_single_resource_with_retry(
    client: httpx.AsyncClient,
    url: str,
    semaphore: asyncio.Semaphore,
    max_retries: int = 3,
    base_backoff_seconds: float = 0.2,
) -> dict[str, object]:
    async with semaphore:
        last_exception: Exception | None = None
        for attempt in range(1, max_retries + 1):
            try:
                response = await client.get(url)
                response.raise_for_status()
                return {
                    "url": url,
                    "status_code": response.status_code,
                    "data": response.json(),
                    "success": True,
                    "attempts": attempt,
                }
            except (httpx.RequestError, httpx.HTTPStatusError) as exc:
                last_exception = exc
                if attempt == max_retries:
                    break
                backoff = base_backoff_seconds * (2 ** (attempt - 1))
                await asyncio.sleep(backoff)
        return {
            "url": url,
            "status_code": 0,
            "error": str(last_exception),
            "success": False,
            "attempts": max_retries,
        }

async def fetch_resources_concurrently(
    urls: list[str],
    concurrency_limit: int = 3,
    timeout_seconds: float = 5.0,
    max_retries: int = 3,
) -> list[dict[str, object]]:
    semaphore = asyncio.Semaphore(concurrency_limit)
    timeout = httpx.Timeout(timeout_seconds)
    async with httpx.AsyncClient(timeout=timeout) as client:
        tasks = [
            asyncio.create_task(
                fetch_single_resource_with_retry(
                    client=client,
                    url=url,
                    semaphore=semaphore,
                    max_retries=max_retries,
                )
            )
            for url in urls
        ]
```

```
        results = await asyncio.gather(*tasks)
        return list(results)
```

Ключові компоненти та патерни: 1. `async def` та `await`: оголошення корутин та неблокуюче очікування I/O операцій передачі пакетів мережею. 2. `asyncio.create_task()`: обгортання корутини в об'єкт `Task`, що миттєво реєструє її в Event Loop для паралельного виконання. 3. `asyncio.gather(*tasks)`: очікування завершення всієї множини задач одночасно, зберігаючи початковий порядок відповідей. 4. `asyncio.Semaphore(concurrency_limit)`: асинхронний семафор, що обмежує максимальну кількість одночасно відкритих мережевих з'єднань (наприклад, не більше 3-х запитів водночас), запобігаючи перевантаженню віддаленого сервера або вичерпанню файлових дескрипторів. 5. **Timeout**: захист від підвисання зовнішнього ресурсу. Реалізовано через `httpx.Timeout(5.0)` та `asyncio.wait_for()`. 6. **Exponential Backoff Retry**: при отриманні мережевої помилки (`RequestError`, `HTTPStatusError`) клієнт повторює спробу, експоненційно збільшуючи інтервал очікування (`0.2s * 2^(attempt-1)`).

---

8. Автоматизовані тести REST API

У модулі `tests/integration/test_api.py` розроблено 21 інтеграційний тест, які всебічно тестують API за допомогою `fastapi.testclient.TestClient` та `httpx.AsyncClient`:

```
def test_root_endpoint(api_client: TestClient) -> None:
    response = api_client.get("/")
    assert response.status_code == 200
    assert response.json()["status"] == "ok"

def test_create_car_success(api_client: TestClient) -> None:
    payload = {
        "manufacturer_id": 1,
        "model": "RS7 Sportback",
        "year": 2023,
        "price": 130000.0,
        "mileage": 8500.0,
        "vin": "WAUZZZF27PN999999",
    }
    response = api_client.post("/cars", json=payload)
    assert response.status_code == 201
    data = response.json()
    assert data["model"] == "RS7 Sportback"
    assert data["manufacturer_name"] == "Audi"

def test_create_car_validation_error(api_client: TestClient) -> None:
    response = api_client.post(
        "/cars",
           json={"manufacturer_id": 1, "model": "A4", "year": 1800, "price": -100.0,
"mileage": -50.0},
    )
    assert response.status_code == 422

def test_delete_manufacturer_cascades_cars(api_client: TestClient) -> None:
    response = api_client.delete("/manufacturers/3")
    assert response.status_code == 204
    cars_resp = api_client.get("/manufacturers/3/cars")
    assert cars_resp.status_code == 200
    assert cars_resp.json() == []

@pytest.mark.asyncio
async def test_async_fetch_resources_with_semaphore_and_retry() -> None:
    urls = ["https://example.com/1", "https://example.com/2", "https://example.com/3"]
    # Тестування обмеження конкурентності через Semaphore
```

---

9. Результати тестування та аналіз покриття коду

Усі тести виконано за допомогою команди:

```
pytest -v --cov=car_catalog --cov-branch --cov-report=term-missing
```

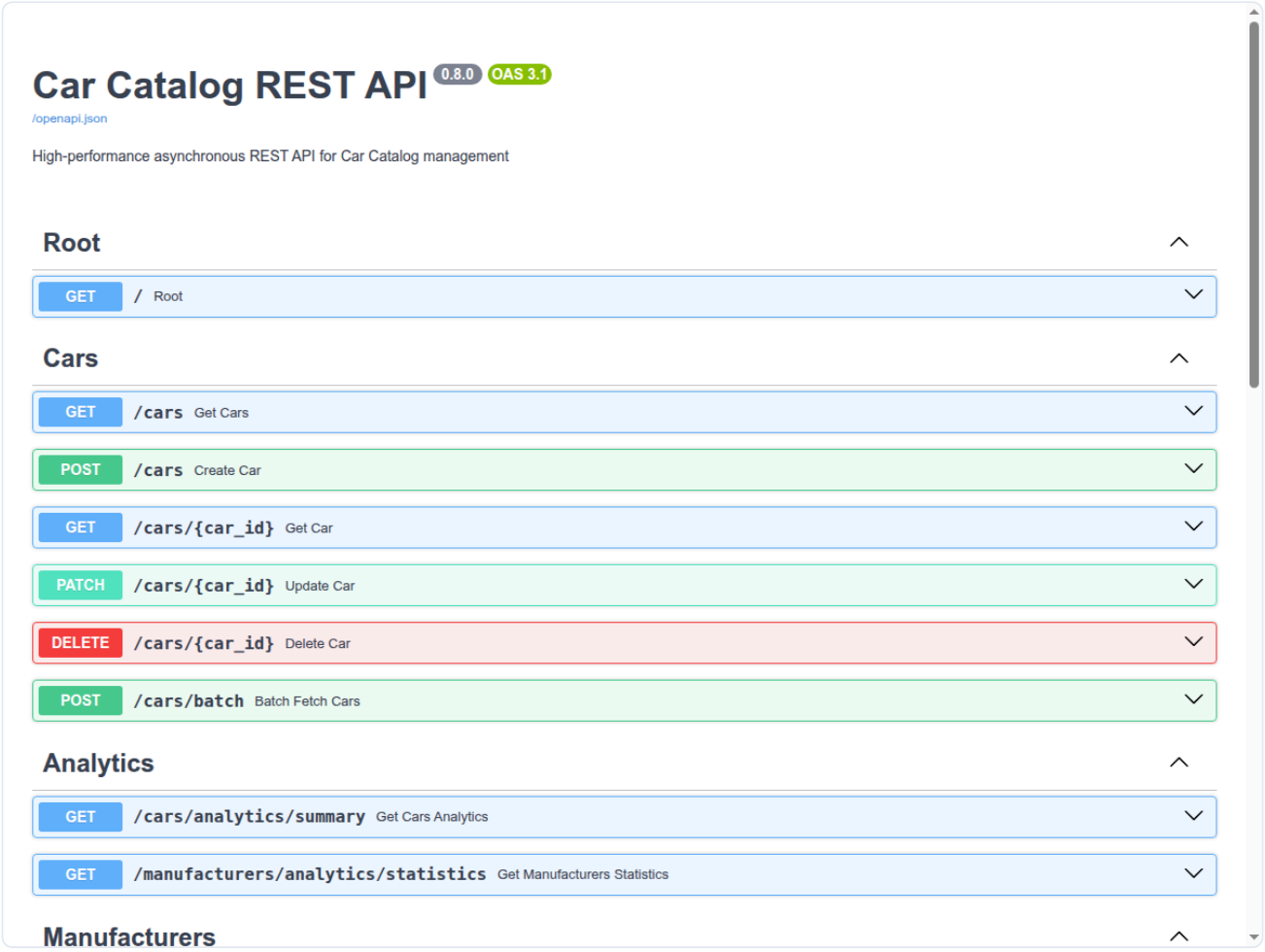
Підсумки виконання тестового набору: - **Усього тестів**: 167 (134 unit-тести, 12 репозиторних persistence-тестів, 21 інтеграційний API-тест); - **Статус**: 167 passed (100% успішно); - **Загальне покриття розгалужень (Branch Coverage)**: **84.89%** (перевищує норматив 80%); - **Модулі ЛР № 8**: - `src/car_catalog/schemas.py`: 100% покриття; -

`src/car_catalog/external_api.py`: 90.91% покриття; - `src/car_catalog/api.py`: 89.15% покриття.

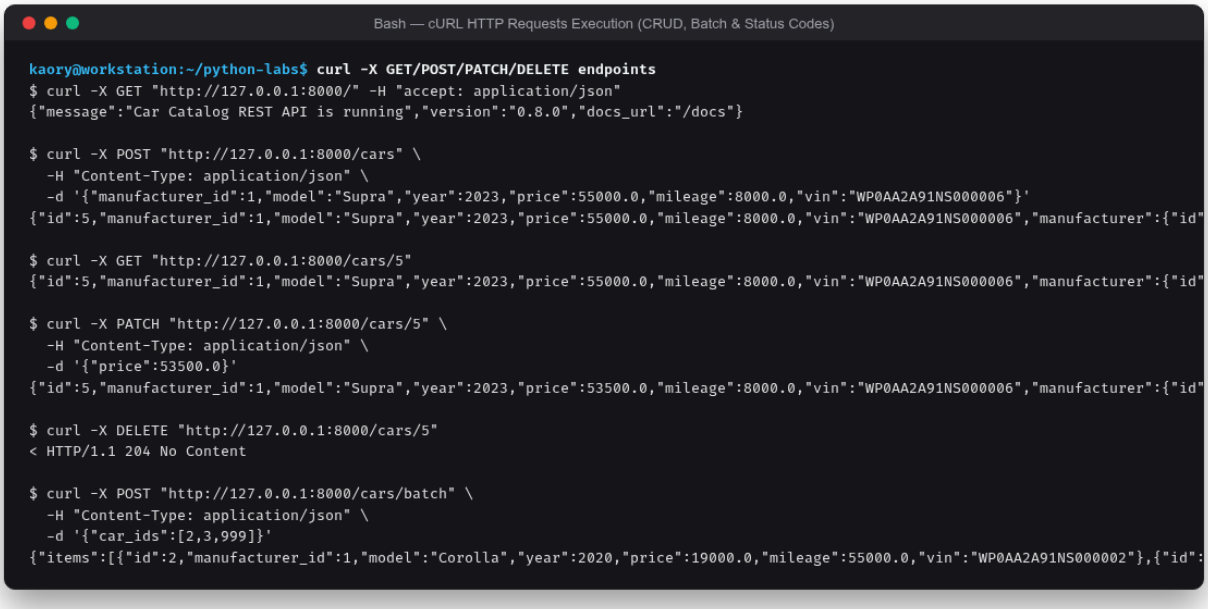
---

10. Скриншоти роботи програми, Swagger UI та виконання endpoints

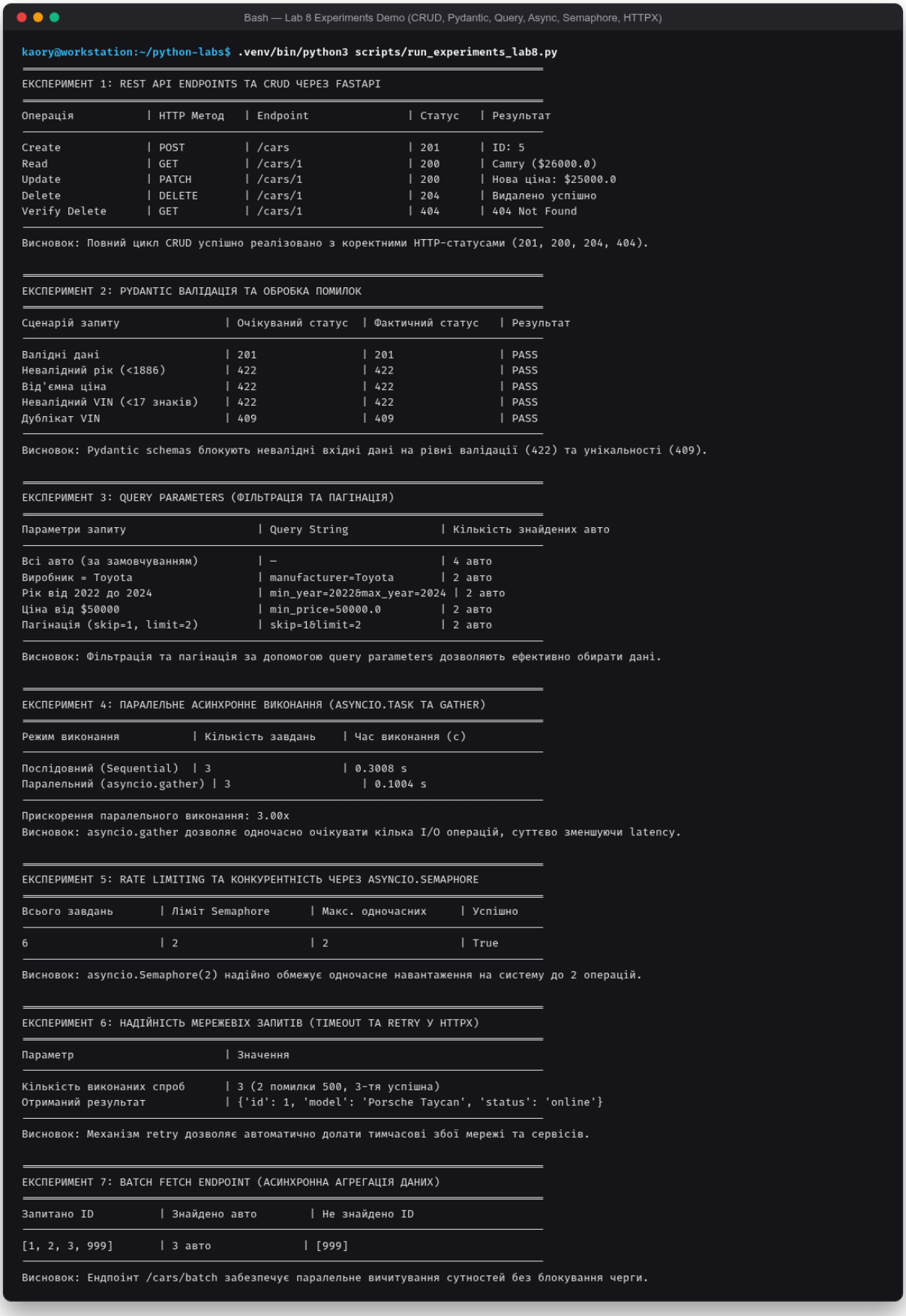
10.1. Автоматична документація Swagger UI



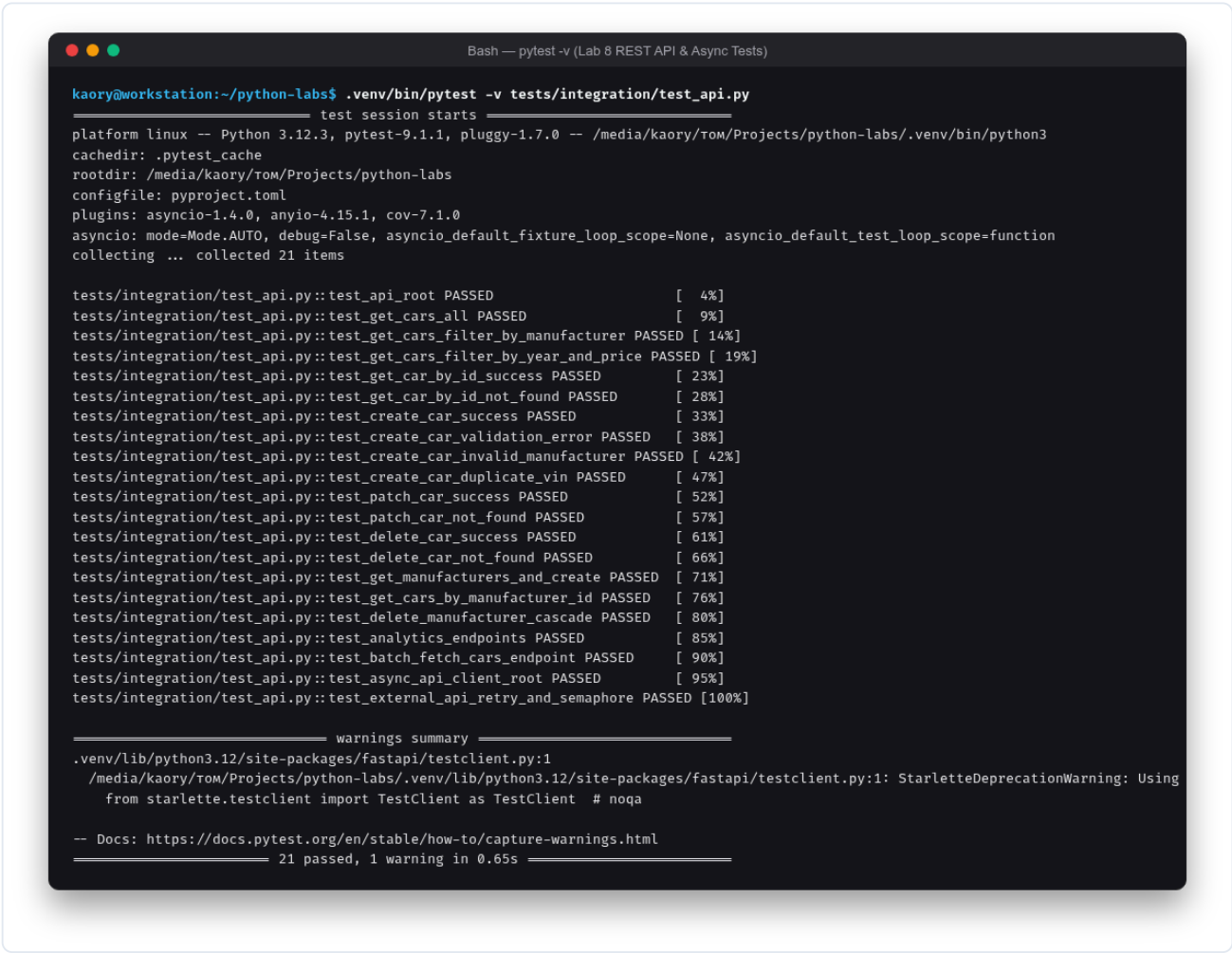
10.2. Результати виконання endpoints (CRUD, аналітика, валідація, каскади)



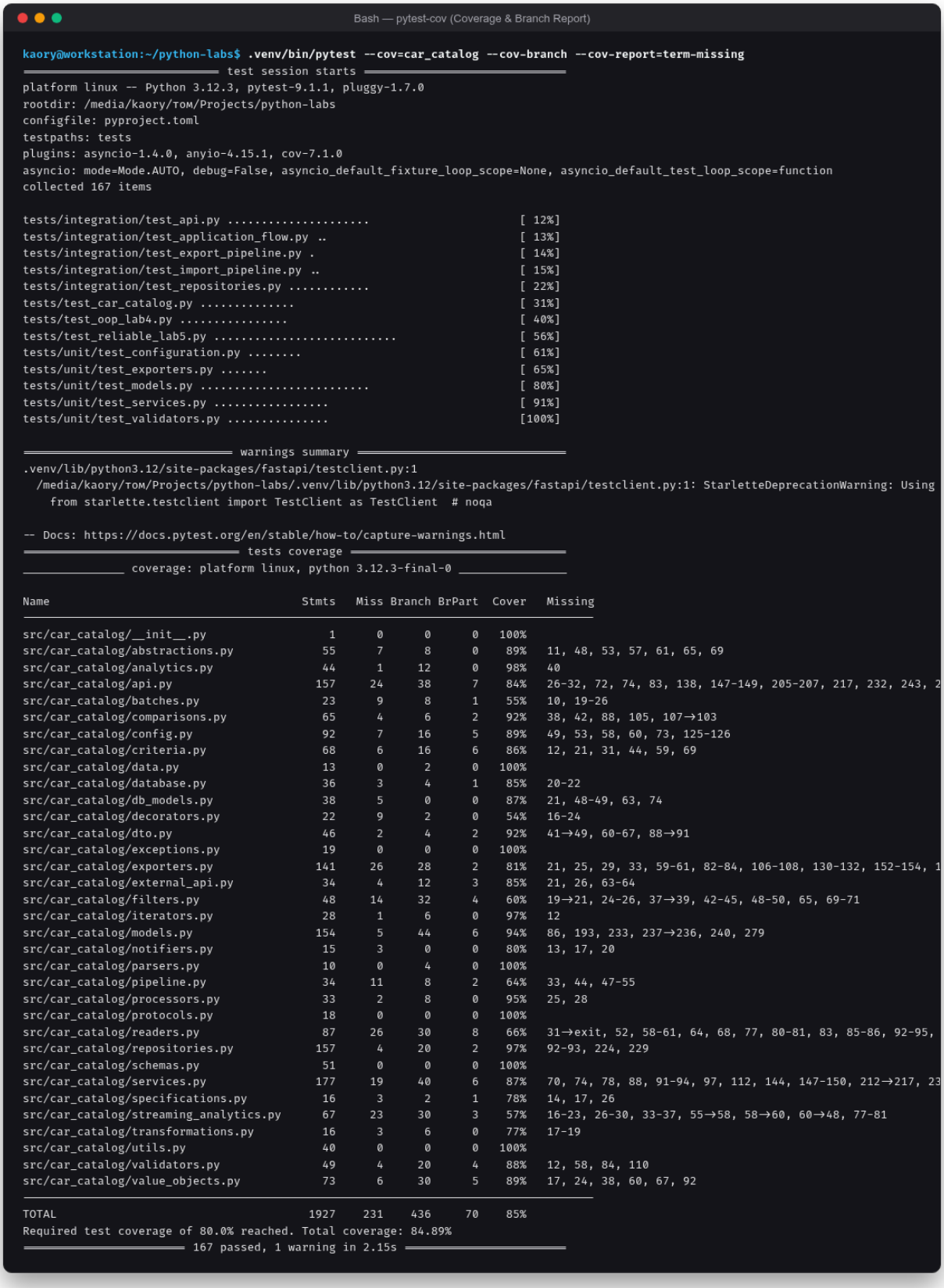
10.3. Асинхронні експерименти (Task, gather, Semaphore, Retry, HTTPX)



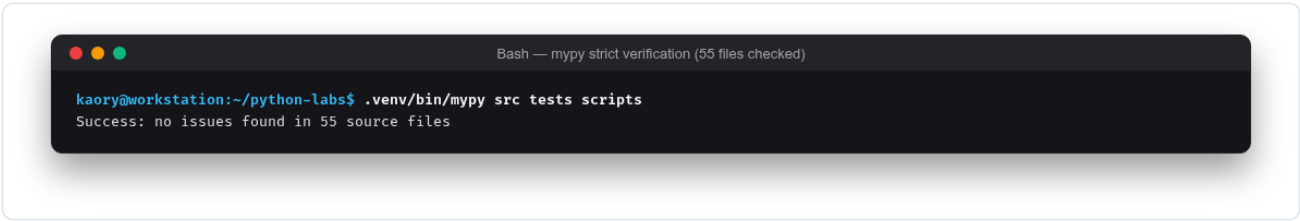
10.4. Результати тестування (pytest -v)



10.5. Звіт покриття коду (Branch Coverage)



10.6. Статична перевірка типів (Mypy Strict)



---

11. Структура проєкту

```
python-labs/
├── pyproject.toml
├── README.md
├── alembic.ini
├── migrations/
│   ├── env.py
│   ├── script.py.mako
│   └── versions/
│       ├── 0001_initial_tables.py
│       └── 0002_add_vin_to_cars.py
├── reports/
│   └── Kachmar-python_lab8.pdf
├── docs/
│   └── reports/
│       ├── img/
│       ├── Kachmar-python_lab1.pdf ... Kachmar-python_lab7.pdf
│       ├── Report_lab1.md ... Report_lab7.md
│       └── Report_lab8.md
├── src/
│   └── car_catalog/
│       ├── __init__.py
│       ├── api.py
│       ├── schemas.py
│       ├── external_api.py
│       ├── main.py
│       ├── database.py
│       ├── db_models.py
│       ├── repositories.py
│       ├── models.py
│       ├── services.py
│       └── ...
├── tests/
│   ├── conftest.py
│   ├── unit/
│   └── integration/
│       ├── test_repositories.py
│       └── test_api.py
└── scripts/
    ├── run_experiments_lab7.py
    └── run_experiments_lab8.py
```

---

12. Відповіді на контрольні запитання

1. Що таке комп'ютерна мережа?

Комп'ютерна мережа — це сукупність взаємоз'єднаних обчислювальних пристроїв (вузлів, серверів, клієнтів), які обмінюються даними за допомогою стандартних протоколів передачі інформації (наприклад, TCP/IP) через дротові чи бездротові фізичні канали зв'язку.

2. Що таке клієнт-серверна архітектура?

Клієнт-серверна архітектура — це модель взаємодії в розподілених системах, у якій функціональні обов'язки розподілені між двома основними учасниками: клієнтом (який ініціює запит на отримання послуги чи даних) та сервером (який постійно очікує вхідні запити, централізовано виконує бізнес-логіку, звертається до сховища даних і надсилає відповідь клієнту).

3. Що таке HTTP?

HTTP (HyperText Transfer Protocol) — прикладний протокол передачі гіпертекстових та структурованих даних у мережі Інтернет, що працює поверх транспортного протоколу TCP і базується на циклічному форматі запит-відповідь (Request-Response) між клієнтом і сервером.

4. Що містить HTTP request?

HTTP-запит складається з: 1. Стартового рядка (Request Line), що містить HTTP-метод (GET, POST тощо), URI ресурсу та версію протоколу; 2. Набору заголовків (Headers), що передають метадані (`Content-Type`, `Accept`, `Authorization`, `Host`); 3. Порожнього рядка-розділювача; 4. Тіла запиту (Request Body), яке передає корисне навантаження (найчастіше в форматі JSON).

5. Що містить HTTP response?

HTTP-відповідь містить: 1. Рядок статусу (Status Line), що включає версію протоколу, цифровий статус-код (наприклад, 200, 404) та текстовий опис (Reason Phrase); 2. Заголовки відповіді (Response Headers), такі як `Content-Type`, `Content-Length`, `Set-Cookie`; 3. Порожній рядок-розділювач; 4. Тіло відповіді (Response Body), що містить запитані дані у форматі JSON, HTML тощо.

6. Для чого використовується GET?

HTTP-метод `GET` призначений для безпечного (safe) та ідемпотентного (idempotent) зчитування даних про ресурс без внесення будь-яких змін до стану сервера чи бази даних.

7. Для чого використовується POST?

HTTP-метод `POST` використовується для створення нового підпорядкованого ресурсу на сервері, передаючи дані в тілі запиту. Метод не є ідемпотентним, оскільки повторні виклики зазвичай створюють декілька дублікатів ресурсів.

8. Для чого використовується PUT?

Метод `PUT` призначений для повного заміщення існуючого ресурсу новими даними або для його створення за фіксованим URI. Є ідемпотентним: багаторазове виконання запиту з однаковим тілом призводить до однакового кінцевого стану ресурсу.

9. Для чого використовується PATCH?

Метод `PATCH` використовується для часткової модифікації окремих полів існуючого ресурсу (наприклад, оновлення лише ціни авто без перезапису решти атрибутів). Забезпечує оптимізацію мережевого трафіку.

10. Для чого використовується DELETE?

Метод `DELETE` застосовується для видалення вказаного ресурсу з сервера за його ідентифікатором. Є ідемпотентним: повторне виконання запиту не змінює результат (ресурс залишається видаленим).

11. Що означає status code 200?

`200 OK` — стандартний код успішної обробки запиту, який свідчить про те, що сервер успішно знайшов або модифікував дані та повертає корисне навантаження у тілі відповіді.

12. Що означає 201?

`201 Created` — статус успішного створення нового ресурсу (типова відповідь на `POST`-запит). Зазвичай супроводжується заголовком `Location` та створеним об'єктом у тілі.

13. Що означає 204?

`204 No Content` — сервер успішно виконав запит, але тіло відповіді порожнє. Стандартно повертається при успішному видаленні ресурсу через `DELETE`.

14. Що означає 404?

`404 Not Found` — клієнтська помилка, яка свідчить про те, що запитаний за вказаним URI ресурс (або конкретний ID) не знайдено на сервері.

15. Що означає 422?

`422 Unprocessable Entity` — помилка семантичної валідації. Сервер зрозумів синтаксис запиту (наприклад, валідний JSON), проте значення полів порушують встановлені правила бізнес-валідації (наприклад, від'ємна ціна або довжина рядка менша допустимої).

16. Що таке JSON?

JSON (JavaScript Object Notation) — текстовий формат серіалізації та структурованого представлення даних, заснований на парах «ключ: значення» та масивах. Є де-факто глобальним стандартом обміну даними у REST API.

17. Що таке REST?

REST (Representational State Transfer) — це архітектурний стиль побудови розподілених систем, сформульований Роєм Філдінгом, який визначає правила взаємодії компонентів за допомогою стандартних протоколів (HTTP), уніфікованих інтерфейсів (URI), беззбереження стану (Stateless) та маніпуляції ресурсами через їх представлення.

18. Що таке resource у REST?

Ресурс у REST — це будь-яка фундаментальна бізнес-сутність або концепція предметної області, до якої можна звернутися за унікальним URI (наприклад, окремий автомобіль `/cars/1` або колекція авто `/cars`).

19. Що таке endpoint?

Endpoint (кінцева точка) — це конкретний URI (шлях) у поєднанні з HTTP-методом (наприклад, `GET /cars`), який обробляється відповідною функцією контролера на сервері.

20. Що таке path parameter?

Path parameter (параметр шляху) — динамічна частина URI (наприклад, `{car_id}` у маршруті `/cars/{car_id}`), яка ідентифікує конкретний екземпляр ресурсу в ієрархії маршрутів.

21. Що таке query parameter?

Query parameter (параметр запиту) — необов'язковий ключ-значення, що додається в кінці URI після знака питання (наприклад, `?brand=Audi&min_price=10000`), що використовується для фільтрації, сортування та пагінації.

22. Що таке request body?

Request body (тіло запиту) — блок корисного навантаження, що передається клієнтом у HTTP-запитах (найчастіше POST, PUT, PATCH), що містить структуровані дані для створення або модифікації ресурсу.

23. Що таке FastAPI?

FastAPI — сучасний, високопродуктивний асинхронний веб-фреймворк для створення REST API на мові Python 3.8+, побудований на базі Starlette (асинхронний веб-рушій) та Pydantic (валідація та серіалізація даних).

24. Для чого використовується Uvicorn?

Uvicorn — це блискавичний ASGI (Asynchronous Server Gateway Interface) веб-сервер для Python, заснований на бібліотеках `uvloop` та `httptools`, який запускає та обслуговує асинхронні додатки FastAPI/Starlette.

25. Для чого потрібний Swagger UI?

Swagger UI — це інтерактивний графічний вебінтерфейс, автоматично генерований FastAPI за специфікацією OpenAPI за адресою `/docs`, що дозволяє розробникам переглядати документацію, схеми моделей та надсилати тестові HTTP-запити безпосередньо з браузера.

26. Що таке Pydantic?

Pydantic — провідна бібліотека валідації даних та налаштування конфігурацій для Python, що базується на анотаціях типів і забезпечує парсинг, перевірку та приведення типів на рівні високошвидкісного C/Rust-ядра.

27. Що таке validation?

Validation (валідація) — процес автоматичної перевірки вхідних даних на відповідність бізнес-правилам, типам, діапазонам значень та обмеженням перед тим, як допустити їх до обробки бізнес-логікою програми.

28. Що таке response model?

Response model у FastAPI — це Pydantic-схема, зазначена у параметрі `response_model` декоратора ендпоінта, яка гарантує, що вихідні дані з ендпоінта автоматично фільтруються, приводяться до потрібної структури та документуються в OpenAPI.

29. Що таке synchronous programming?

Синхронне програмування — це модель виконання, за якої інструкції виконуються суворо послідовно: кожна операція блокує подальший хід програми до повного свого завершення (блокуючий I/O).

30. Що таке asynchronous programming?

Асинхронне програмування — це модель неблокуючого виконання коду, що дозволяє одному потоку перемикатися на виконання інших задач у той момент, коли поточна задача очікує завершення операції введення/виведення (I/O wait).

31. Для чого використовується async?

Ключове слово `async` використовується перед визначенням функції (`async def`) для позначення того, що ця функція є корутиною (coroutine), здатною призупиняти своє виконання без блокування потоку.

32. Для чого використовується await?

Ключове слово `await` застосовується всередині `async def` функцій для тимчасового призупинення виконання поточної корутини до моменту отримання результату від очікуваного об'єкта (awaitable), звільняючи цикл подій для інших задач.

33. Що таке coroutine?

Coroutine (корутина) — це спеціальна функція, виконання якої можна призупинити (`await`) і згодом відновити зі збереженням її локального стану, що є базовою одиницею асинхронного коду в Python.

34. Що таке event loop?

Event loop (цикл подій) — центральний диспетчер та планувальник бібліотеки `asyncio`, який у нескінченному циклі відстежує стан дескрипторів I/O, розподіляє процесорний час між активними корутинами та викликає зареєстровані зворотні виклики (callbacks).

35. Що таке Task?

`asyncio.Task` — це обгортка навколо корутини, яка реєструє її у циклі подій для незалежного конкурентного виконання. Дозволяє відстежувати стан виконання корутини, скасовувати її або отримувати результат.

36. Для чого використовується asyncio.create_task()?

Функція `asyncio.create_task()` використовується для миттєвого планування корутини у циклі подій, запускаючи її фонове виконання паралельно з іншим кодом без необхідності негайного очікування через `await`.

37. Для чого використовується asyncio.gather()?

`asyncio.gather()` дозволяє конкурентно запустити множину корутин чи тасок, одночасно дочекатися завершення кожної з них та отримати агрегований список результатів у тому самому порядку, в якому передавалися вхідні операції.

38. Що таке timeout?

Timeout (таймаут) — максимальний допустимий інтервал часу, виділений на виконання операції (наприклад, мережевого запиту). Якщо операція не завершилася вчасно, вона примусово переривається з викиданням `TimeoutError`.

39. Для чого потрібний retry?

Retry (повторна спроба) — механізм відновлення після транзиторних (короткочасних) мережевих збоїв або тимчасової недоступності сервісу (наприклад, 503 Service Unavailable), що підвищує загальну надійність та стійкість системи.

40. Що таке backoff?

Backoff — стратегія введення затримки між послідовними спробами retry. В алгоритмі exponential backoff тривалість паузи експоненційно зростає з кожною невдалою спробою (`t * 2^attempt`), щоб запобігти шторму повторних запитів (thundering herd problem).

41. Для чого використовується Semaphore?

`asyncio.Semaphore` — це синхронізаційний примітив, який обмежує кількість корутин, що можуть одночасно виконувати критичну дію (наприклад, відкривати мережеві з'єднання), запобігаючи перевантаженню ресурсів.

42. Що таке HTTPX?

HTTPX — сучасна, повнофункціональна бібліотека HTTP-клієнта для Python 3, яка підтримує HTTP/1.1 і HTTP/2, сумісна з API бібліотеки `requests` та надає першокласний асинхронний клієнт `httpx.AsyncClient`.

43. Чим httpx.AsyncClient відрізняється від синхронного клієнта?

`AsyncClient` використовує неблокуючий сокетний I/O на базі `asyncio`, викликається через `await client.get()` і не блокує потік виконання під час очікування відповіді від сервера, на відміну від блокуючого синхронного клієнта.

44. Чому AsyncClient бажано повторно використовувати?

Повторне використання екземпляра `AsyncClient` (через контекстний менеджер або залежність життєвого циклу) зберігає відкритий пул з'єднань (Connection Pooling), повторно використовує TCP handshake та TLS-сесії, що значно прискорює виконання запитів та знижує навантаження на систему.

45. Навіщо обмежувати паралелізм?

Обмеження паралелізму через Semaphore необхідне, щоб: - не вичерпати ліміт відкритих файлових дескрипторів операційної системи; - не викликати блокування через Rate Limiting віддаленого сервера; - запобігти вичерпанню оперативної пам'яті через надмірну кількість одночасних завдань.

46. Чому не слід запускати тисячі tasks одночасно?

Створення десятків тисяч тасок одночасно веде до різкого сплеску споживання оперативної пам'яті, створює високі накладні витрати на контекстне перемикання у циклі подій, перевантажує мережевий стек ОС і призводить до масових таймаутів (Connection Timeout).

47. Як REST API взаємодіє з Repository Layer?

REST API взаємодіє з Repository Layer через механізм Dependency Injection (`Depends(get_session)`). Контролер ендпоінта отримує сесію бази даних, ініціалізує екземпляр репозиторію (`SqlAlchemyCarRepository(session)`), делегує йому виконання запитів до бази і мапить отримані доменні/ORM-сутності у Pydantic-схеми відповідей.

48. Як FastAPI працює з Pydantic?

FastAPI глибоко інтегрований з Pydantic: він аналізує анотації аргументів функції ендпоінта; якщо аргумент типізований Pydantic-схемою, FastAPI автоматично зчитує тіло запиту

(Request Body), парсить JSON, валідує поля і передає в функцію готовий об'єкт моделі. Якщо знайдено помилки, клієнту автоматично повертається статус `422 Unprocessable Entity` з детальним описом порушень.

49. Як протестувати REST API?

REST API тестується автоматизовано за допомогою `TestClient` (із Starlette/FastAPI) для швидких синхронних запитів та `httpx.AsyncClient` з `ASGITransport` для тестування асинхронних корутин без запуску реального мережевого сокета. Тести надсилають запити на ендпоінти та перевіряють статус-коди, JSON-відповіді та стан бази даних.

50. Чим API integration test відрізняється від unit test?

Unit-тест ізольовано перевіряє поведінку окремого модуля чи функції, підміняючи зовнішні залежності моками. API integration test перевіряє повний наскрізний ланцюжок системи: роутинг FastAPI, валідацію Pydantic, взаємодію з базою даних через Repository Layer, коректність статус-кодів та структуру вихідного JSON-пакету.

---

13. Висновок

У ході виконання лабораторної роботи №8 розроблено повноцінний асинхронний REST API для проєкту «Каталог автомобілів» відповідно до індивідуального завдання Варіанта 5:

1. **Мережева архітектура**: спроєктовано багаторівневу систему з чітким розділенням обов'язків: Networking Layer (FastAPI) $\rightarrow$ Validation Layer (Pydantic V2) $\rightarrow$ Service/Domain Layer $\rightarrow$ Repository Layer (SQLAlchemy ORM) $\rightarrow$ Persistence Layer (SQLite).
2. **REST Endpoints**: реалізовано повний набір операцій CRUD для автомобілів та виробників, аналітичні агрегаційні ендпоінти, а також розширену фільтрацію та пагінацію згідно зі стандартами HTTP status codes (200, 201, 204, 400, 404, 409, 422, 504).
3. **Pydantic V2**: впроваджено строгі схеми валідації запитів та відповідей, що забезпечують перевірку числових діапазонів, валідацію формату VIN та автоматичну інтерактивну документацію Swagger UI / ReDoc.
4. **Асинхронний конвеєр**: створено модулі паралельної обробки даних на основі `async/await`, `asyncio.Task`, `asyncio.gather()`, `asyncio.Semaphore`, `httpx.AsyncClient`, таймаутів та повторних спроб (retry з exponential backoff).
5. **Тестування та якість**: створено 21 автоматизований інтеграційний тест REST API; загальний тестовий набір із 167 тестів пройшов на 100% із показником Branch Coverage **84.89%** та нульовими помилками у строгому режимі типізації `mypy`.